

Lyra Project Master Plan: 從根基到星系

版本: Final Draft (Flutter Edition) **狀態:** Ready for Development **核心哲學:** Bit-perfect (位元完美), Content-Addressable (內容定址), Database-Centric (資料庫主導)

1. 專案層級定義 (Hierarchy of Scope)

在開始寫 Code 之前，必須嚴格遵守以下的分層。這能防止你在開發初期迷失方向。

```
graph TD
  A[The Root: 根基] --> B[The Core: 核心 (v0.1 MVP)]
  B --> C[The Expansion: 擴充 (v0.2 - v1.0)]

  style A fill:#f9f,stroke:#333,stroke-width:4px
  style B fill:#bbf,stroke:#333,stroke-width:2px
  style C fill:#ddd,stroke:#333,stroke-dasharray: 5 5
```

1.1 The Root: 根基 (不可動搖的原則)

這些是 Lyra 的靈魂。如果拿掉這些，Lyra 就只是普通的播放器。

1. CAS (Content-Addressable Storage):

- **原則:** 檔名即雜湊 (Filename is Hash)。
- **實作:** 使用 `ffmpeg -f md5` 計算解碼後的音訊流雜湊。
- **意義:** 確保「音質內容」的唯一性。無論 Tag 怎麼改，只要聲音沒變，Hash 就不變。

2. Database as Source of Truth:

- **原則:** 一旦匯入，所有 Metadata (歌名、歌手) 以資料庫為準，不回寫檔案。
- **實作:** SQLite 是唯一的真理。

3. Immutability (不可變性):

- **原則:** 儲存庫 (/objects/) 裡的檔案一旦寫入，就視為唯讀。要修改音訊內容 = 產生新檔案 = 產生新 Hash。

1.2 The Core: 核心 (v0.1 MVP - 初期目標)

這是在前 6 週唯一要做的事。目標是「單機可用」。

- **平台:** Linux (Ubuntu) & Windows (你有的環境)。
- **功能:** 匯入 (Import)、儲存 (Store)、列出 (List)、播放 (Play)。
- **技術:** * **Backend:** C++20, SQLite, FFmpeg, TagLib.

- **Frontend:** Flutter (Dart).
- **Bridge:** Dart FFI (Foreign Function Interface).
- **限制:** 不聯網、不跑 Server Daemon、不跑 Python、不跑 LLM。

1.3 The Expansion: 擴充 (後期目標 - 未來再做)

這些是「想要」但初期做會導致專案失敗的功能。

- **網路化:** gRPC Server, Remote Client, Mobile App.
- **同步:** 雙向同步, Action Queue, Tombstones.
- **智慧化:** yt-dlp 下載器, LLM 標籤分析.
- **轉碼:** 即時 Opus 轉碼串流.

2. 系統詳細規格 (System Specification)

2.1 檔案系統結構 (The Root Level)

無論是單機版還是伺服器版，檔案結構永遠長這樣：

- **Root Path:** * Linux: ~/Music/LyraRepo/ (開發期), /var/lib/lyra/ (完成期)
 - Windows: D:\LyraRepo\
- **Object Storage:**
 - .../objects/{hash[0:2]}/{hash[2:4]}/{hash}.{ext}
 - 範例: .../objects/a1/b2/a1b2c3d4e5... .flac
- **Database:**
 - lyra.db (SQLite) 位於 Root Path 根目錄。

2.2 資料庫 Schema (The Core Level - v0.1)

為了快速啟動，我們使用 **v0.1 精簡版 Schema**，但保持欄位命名與未來 v2.6 相容。

```
-- 1. 來源紀錄 (溯源用)
CREATE TABLE Source_Data (
    source_id INTEGER PRIMARY KEY AUTOINCREMENT,
    original_path TEXT NOT NULL, -- 記錄匯入前的原始路徑 (D:\Music\...)
    date_added INTEGER NOT NULL -- Unix Timestamp
);

-- 2. 歌曲資訊 (核心表)
-- 在 v0.1 中，我們暫時將 Work(作品) 與 Track(錄音) 合併，簡化邏輯
CREATE TABLE Tracks (
    track_uuid TEXT PRIMARY KEY, -- UUID v4 (如 "550e8400-...")
    title TEXT, -- 歌名
    artist TEXT, -- 歌手
```

```

        album TEXT,                -- 專輯
        primary_file_hash TEXT NOT NULL, -- FK -> 指向 Files 表 (Hex String)
        source_id INTEGER,          -- FK -> 指向 Source_Data
        FOREIGN KEY(primary_file_hash) REFERENCES Files(file_hash),
        FOREIGN KEY(source_id) REFERENCES Source_Data(source_id)
    );

-- 3. 實體檔案 (CAS 索引)
CREATE TABLE Files (
    file_hash TEXT PRIMARY KEY,      -- ffmpeg md5 (Hex String)
    format TEXT,                    -- "flac", "mp3"
    size_bytes INTEGER,
    duration_sec INTEGER
);

-- 4. 封面圖片 (簡化版 - 僅存原始圖)
CREATE TABLE Images (
    image_hash TEXT PRIMARY KEY,     -- sha256 (Hex String)
    format TEXT                     -- "jpg", "png"
);

-- v0.1 暫不實作 Tracks 與 Images 的關聯，先求有聲音

```

3. 開發實作計畫 (Implementation Plan)

這是給高二學生的 C++ 學習與開發衝刺表。

階段一：地基與雜湊 (Week 1-2)

目標：寫出一個能算 Hash 並搬運檔案的 **C++ 動態庫 (Shared Library)**。

1. 環境建置:

- 安裝 CMake, Ninja, vcpkg。
- **重點:** CMakeLists.txt 必須設定 `add_library(lyra SHARED ...)`。這會產生 `liblyra.so` (Linux) 或 `lyra.dll` (Windows)，這是 Flutter 呼叫的對象。

2. C-API 封裝 (關鍵):

- 因為 Flutter (Dart) 呼叫 C++ 類別很麻煩，你需要寫 `extern "C"` 介面。
- 範例: `extern "C" void lyra_init(const char* db_path);`
- 範例: `extern "C" char* lyra_import_file(const char* file_path);`

3. Hash 與搬運:

- 實作 `std::string compute_audio_hash(path)` (內部 C++ 邏輯)。
- 實作檔案複製與目錄建立。

階段二：資料庫與讀取 (Week 3-4)

目標：C++ Core 完成資料庫操作，並暴露 C 介面給外界。

1. Metadata 讀取:

- 引入 TagLib 。
- 讀取匯入檔案的 Title, Artist 。

2. 資料庫串接:

- 使用 Transaction (交易) 確保原子性。
- **C-API 擴充:** 實作 `lyra_get_all_tracks()`，這需要設計一個 C struct 來回傳陣列資料給 Dart 。

3. 產出: 雖然還沒有 GUI，但你可以寫一個簡單的 `main.cpp` 去呼叫這些 `.so` 裡的函數來測試功能。

階段三：Flutter GUI 整合 (Week 5-6)

目標：使用 Flutter 畫出介面，並透過 FFI 讓它發出聲音。

1. Flutter 環境:

- `flutter create lyra_desktop --platforms=linux,windows`
- 設定 `pubspec.yaml` 引入 `ffi` 套件。

2. FFI 橋接 (The Glue):

- 在 Dart 中定義對應 C++ 的函數簽名。
- `DynamicLibrary.open('liblyra.so')` 載入你的核心。
- 呼叫 `lyra_init()` 和 `lyra_get_all_tracks()` 把資料顯示在 `ListView` 上。

3. 播放引擎 (Backend):

- 在 C++ 端引入 `miniaudio` 。
- 暴露 `extern "C" void lyra_play(const char* uuid);`
- Flutter 按鈕 `onPressed` -> 呼叫 FFI -> C++ 播放聲音。

4. 未來擴充路線 (The Roadmap to v2.6)

完成上述 v0.1 後，你才有資格進入這裡。

1. v0.2 (Mobile Adaptation):

- 因為 UI 已經是 Flutter，你只需要重新編譯 C++ Core 為 Android `.so`，UI 幾乎不用改就能在手機跑。

2. v0.3 (Daemon化):

- 將 C++ Core 獨立成 Daemon，Flutter 改用 gRPC 呼叫 (替代 FFI)，實現前後端分離。

3. v1.0 (The Sync):

- 加入 `Pending_Actions` 表，開始處理雙向同步。

5. 給開發者的備忘錄 (Developer Notes)

- **FFI 是雙面刃:** FFI 很強大，但如果在 C++ 端 `segmentation fault`，整個 Flutter App 會直接閃退，沒錯誤訊息。務必在 C++ 端做好 `try-catch`，不要讓 Exception 拋給 Dart。
- **記憶體管理:** 如果 C++ 回傳字串指標 (`char*`) 給 Dart，Dart 用完後誰負責釋放記憶體？通常需要在 C++ 提供 `lyra_free_string(char* str)` 讓 Dart 呼叫。
- **關於 UUID:** C++ 標準庫目前對 UUID 支援不全，建議使用 `stduuid` 庫。
- **保持簡單:** 如果遇到一個功能覺得很難寫，**砍掉它**。v0.1 的重點是架構，不是功能。

Good luck, Commander. 願你的編譯器永遠 0 Error.